



Faculty of Engineering
Programming for Problem Solving
using C
Laboratory Manual

NAME:

ENROLLMENT NUMBER:

SUBJECT NAME:

SUBJECT CODE: BRANCH:

SEMESTER: ACADEMIC YEAR:



Swaminarayan university
Publication

ISBN Number:



सर्व विद्या अधिपतये

Certificate

This is to certify that Mr. / Ms. / Mrs.
having enrollment number of Semester ME /
BE / DIPLOMA in.....has satisfactorily completed his
term working (Subject name) (subject code:.....)
during academic year at Faculty of Engineering, Swaminarayan University.

Place:

Date:

ASHWINI WALAKE
SUBJECT IN-CHARGE

HOD

DR. BHAVNA SONI
DEAN

List of Practical

Sr. No.	Title	Page No.	Date	Sign
1	Write a program that performs basic input and output operations and displays the use of variables, constants and different data types in C.			
2	Write a program that performs basic arithmetic operations such as addition, subtraction, multiplication, division and modulus using operators and expressions.			
3	Write a program to demonstrate arithmetic, relational, logical, assignment, increment, decrement, conditional and bitwise operators.			
4	Create a menu-driven program using if, if...else, nested if...else, else-if ladder and switch statement.			
5	Write a program using while, do...while and for loops to perform repetitive tasks such as factorial, sum of digits and multiplication table.			
6	Write a program to generate different pattern-based outputs using nested loops.			
7	Write a program to perform various operations on one-dimensional arrays such as insertion, deletion, searching and sorting.			
8	Write a program to perform matrix operations such as addition, subtraction and multiplication using two-dimensional arrays.			
9	Write a program to perform string operations such as length, copy, concatenation and comparison using built-in string functions.			
10	Write a program to perform string manipulation without using built-in string functions.			
11	Develop a program that calculates factorial and Fibonacci series using user-defined functions and			

	recursive functions.			
12	Design a student record system using structures to store and display information such as roll number, name, marks and grade.			
13	Write a program to demonstrate structures, nested structures, arrays within structures and unions.			
14	Write a program that uses pointers with arrays and functions and demonstrates dynamic memory allocation using malloc(), calloc(), realloc() and free().			
15	Write a program to read from and write to files using file handling functions such as fopen(), fclose(), fprintf(), fscanf(), fread() and fwrite().			

Practical-1

AIM: Write a program that performs basic input and output operations and displays the use of variables, constants and different data types in C

CODE:

```
#include <stdio.h>

int main() {
    // Variable declarations
    int age;
    float height;
    char grade;

    // Constant declaration
    const float PI = 3.14159;

    // Input
    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (in meters): ");
    scanf("%f", &height);

    printf("Enter your grade: ");
    scanf(" %c", &grade);

    // Output
    printf("\n--- Student Information ---\n");
    printf("Age   : %d\n", age);
    printf("Height : %.2f meters\n", height);
    printf("Grade  : %c\n", grade);

    // Display constant
    printf("Value of PI (constant): %.5f\n", PI);

    return 0;
}
```

OUTPUT:

```
Enter your age: 20
Enter your height (in meters): 1.75
Enter your grade: A
--- Student Information ---
Age : 20
Height : 1.75 meters
Grade : A
Value of PI (constant): 3.14159
```

Practical-2

AIM : Write a program that performs basic arithmetic operations such as addition, subtraction, multiplication, division and modulus using operators and expressions.

CODE:

```
#include <stdio.h>

int main() {
    int a, b;

    // Input
    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);

    // Arithmetic operations
    printf("\nAddition = %d\n", a + b);
    printf("Subtraction = %d\n", a - b);
    printf("Multiplication = %d\n", a * b);
    printf("Division = %d\n", a / b);
    printf("Modulus = %d\n", a % b);

    return 0;
}
```

OUTPUT:

Enter two integers: 20 5

Addition = 25

Subtraction = 15

Multiplication = 100

Division = 4

Modulus = 0

Practical-3

AIM : Write a program to demonstrate arithmetic, relational, logical, assignment, increment, decrement, conditional and bitwise operators.

CODE:

```
#include <stdio.h>

int main() {
    int a = 10, b = 5, c;

    // Arithmetic Operators
    printf("Arithmetic Operators:\n");
    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);
    printf("a %% b = %d\n\n", a % b);

    // Relational Operators
    printf("Relational Operators:\n");
    printf("a > b = %d\n", a > b);
    printf("a < b = %d\n", a < b);
    printf("a == b = %d\n\n", a == b);

    // Logical Operators
    printf("Logical Operators:\n");
    printf("(a > b && b > 0) = %d\n", (a > b && b > 0));
    printf("(a < b || b > 0) = %d\n", (a < b || b > 0));
    printf("!(a > b) = %d\n\n", !(a > b));

    // Assignment Operator
    c = a;
    c += b;
    printf("Assignment Operator:\n");
    printf("c = %d\n\n", c);

    // Increment and Decrement Operators
    printf("Increment and Decrement Operators:\n");
    printf("++a = %d\n", ++a);
    printf("--b = %d\n\n", --b);
```

```
// Conditional (Ternary) Operator
printf("Conditional Operator:\n");
printf("Larger number = %d\n\n", (a > b) ? a : b);

// Bitwise Operators
printf("Bitwise Operators:\n");
printf("a & b = %d\n", a & b);
printf("a | b = %d\n", a | b);
printf("a ^ b = %d\n", a ^ b);
printf("a << 1 = %d\n", a << 1);
printf("a >> 1 = %d\n", a >> 1);

return 0;
}
```

OUTPUT:

Arithmetic Operators:

a + b = 15

a - b = 5

a * b = 50

a / b = 2

a % b = 0

Relational Operators:

a > b = 1

a < b = 0

a == b = 0

Logical Operators:

(a > b && b > 0) = 1

(a < b || b > 0) = 1

!(a > b) = 0

Assignment Operator:

c = 15

Increment and Decrement Operators:

++a = 11

--b = 4

Conditional Operator:

Larger number = 11

Bitwise Operators:

$a \& b = 0$

$a | b = 15$

$a \wedge b = 15$

$a \ll 1 = 22$

$a \gg 1 = 5$

Practical-4

AIM : Create a menu-driven program using if, if...else, nested if...else, else-if ladder and switch statement.

CODE:

```
#include <stdio.h>

int main() {
    int choice;
    int num1, num2;

    printf("----- MENU -----\\n");
    printf("1. Check Positive or Negative (if\\n");
    printf("2. Check Even or Odd (if...else)\\n");
    printf("3. Find Largest of Two Numbers (nested if...else)\\n");
    printf("4. Find Grade (else-if ladder)\\n");
    printf("5. Arithmetic Operations (switch)\\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice) {

        case 1:
            printf("Enter a number: ");
            scanf("%d", &num1);

            if(num1 > 0)
                printf("%d is Positive.\\n", num1);

            break;

        case 2:
            printf("Enter a number: ");
            scanf("%d", &num1);

            if(num1 % 2 == 0)
                printf("%d is Even.\\n", num1);
            else
                printf("%d is Odd.\\n", num1);

            break;
```

```
case 3:
    printf("Enter two numbers: ");
    scanf("%d%d", &num1, &num2);

    if(num1 != num2) {
        if(num1 > num2)
            printf("%d is Larger.\n", num1);
        else
            printf("%d is Larger.\n", num2);
    }
    else {
        printf("Both numbers are Equal.\n");
    }

    break;

case 4: {
    int marks;
    printf("Enter marks: ");
    scanf("%d", &marks);

    if(marks >= 90)
        printf("Grade A\n");
    else if(marks >= 75)
        printf("Grade B\n");
    else if(marks >= 60)
        printf("Grade C\n");
    else if(marks >= 40)
        printf("Grade D\n");
    else
        printf("Fail\n");

    break;
}

case 5: {
    int op;
    printf("Enter two numbers: ");
    scanf("%d%d", &num1, &num2);

    printf("\n1.Add\n2.Subtract\n3.Multiply\n4.Divide\n");
    printf("Choose operation: ");
    scanf("%d", &op);

    switch(op) {
```

```

        case 1:
            printf("Addition = %d\n", num1 + num2);
            break;
        case 2:
            printf("Subtraction = %d\n", num1 - num2);
            break;
        case 3:
            printf("Multiplication = %d\n", num1 * num2);
            break;
        case 4:
            if(num2 != 0)
                printf("Division = %.2f\n", (float)num1 / num2);
            else
                printf("Division by zero is not possible.\n");
            break;
        default:
            printf("Invalid operation!\n");
    }

    break;
}

default:
    printf("Invalid Choice!\n");
}

return 0;
}

```

OUTPUT:

----- MENU -----

1. Check Positive or Negative (if)
2. Check Even or Odd (if...else)
3. Find Largest of Two Numbers (nested if...else)
4. Find Grade (else-if ladder)
5. Arithmetic Operations (switch)

Enter your choice: 2

Enter a number: 8

8 is Even.

Practical-5

AIM : Write a program using while, do...while and for loops to perform repetitive tasks such as factorial, sum of digits and multiplication table.

CODE:

```
#include <stdio.h>

int main() {
    int choice, n, i, fact = 1, sum = 0, temp;

    printf("----- MENU -----\\n");
    printf("1. Factorial (while loop)\\n");
    printf("2. Sum of Digits (do...while loop)\\n");
    printf("3. Multiplication Table (for loop)\\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice) {

        case 1:
            printf("Enter a number: ");
            scanf("%d", &n);

            i = 1;
            while(i <= n) {
                fact = fact * i;
                i++;
            }

            printf("Factorial of %d = %d\\n", n, fact);
            break;

        case 2:
            printf("Enter a number: ");
            scanf("%d", &n);

            temp = n;
            do {
                sum = sum + (temp % 10);
                temp = temp / 10;
            } while(temp != 0);
```

```
    printf("Sum of digits of %d = %d\n", n, sum);
    break;

    case 3:
        printf("Enter a number: ");
        scanf("%d", &n);

        printf("Multiplication Table of %d:\n", n);
        for(i = 1; i <= 10; i++) {
            printf("%d x %d = %d\n", n, i, n * i);
        }
        break;

    default:
        printf("Invalid Choice!\n");
}

return 0;
}
```

OUTPUT:

Output 1 (Factorial)

----- MENU -----

1. Factorial (while loop)
2. Sum of Digits (do...while loop)
3. Multiplication Table (for loop)

Enter your choice: 1

Enter a number: 5

Factorial of 5 = 120

Output 2 (Sum of Digits)

----- MENU -----

1. Factorial (while loop)
2. Sum of Digits (do...while loop)
3. Multiplication Table (for loop)

Enter your choice: 2

Enter a number: 1234

Sum of digits of 1234 = 10

Output 3 (Multiplication Table)

----- MENU -----

1. Factorial (while loop)
2. Sum of Digits (do...while loop)
3. Multiplication Table (for loop)

Enter your choice: 3

Enter a number: 7

Multiplication Table of 7:

7 x 1 = 7

7 x 2 = 14

7 x 3 = 21

7 x 4 = 28

7 x 5 = 35

7 x 6 = 42

7 x 7 = 49

7 x 8 = 56

7 x 9 = 63

7 x 10 = 70

Practical-6

AIM : Write a program to generate different pattern-based outputs using nested loops.

CODE:

```
#include <stdio.h>

int main() {
    int i, j, n;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    // Pattern 1: Right Triangle
    printf("\nPattern 1:\n");
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    // Pattern 2: Inverted Right Triangle
    printf("\nPattern 2:\n");
    for(i = n; i >= 1; i--) {
        for(j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    // Pattern 3: Number Triangle
    printf("\nPattern 3:\n");
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= i; j++) {
            printf("%d ", j);
        }
        printf("\n");
    }

    return 0;
}
```

OUTPUT:

Enter the number of rows: 5

Pattern 1:

```
*
* *
* * *
* * * *
```

* * * * *

Pattern 2:

* * * * *

* * * *

* * *

* *

*

Pattern 3:

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Practical-7

AIM : Write a program to perform various operations on one-dimensional arrays such as insertion, deletion, searching and sorting.

CODE:

```
#include <stdio.h>

int main() {
    int a[100], n, i, choice, pos, item, found = 0, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("\n1. Insertion\n2. Deletion\n3. Searching\n4. Sorting\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice) {

        case 1:
            printf("Enter position and element to insert: ");
            scanf("%d%d", &pos, &item);

            for(i = n; i >= pos; i--)
                a[i] = a[i - 1];

            a[pos - 1] = item;
            n++;

            printf("Array after insertion:\n");
            for(i = 0; i < n; i++)
                printf("%d ", a[i]);
            break;
```

case 2:

```
printf("Enter position to delete: ");
scanf("%d", &pos);
```

```
for(i = pos - 1; i < n - 1; i++)
    a[i] = a[i + 1];
```

```
n--;
```

```
printf("Array after deletion:\n");
for(i = 0; i < n; i++)
    printf("%d ", a[i]);
break;
```

case 3:

```
printf("Enter element to search: ");
scanf("%d", &item);
```

```
for(i = 0; i < n; i++) {
    if(a[i] == item) {
        found = 1;
        printf("Element found at position %d\n", i + 1);
        break;
    }
}
```

```
if(found == 0)
    printf("Element not found\n");
break;
```

case 4:

```
for(i = 0; i < n - 1; i++) {
    for(int j = i + 1; j < n; j++) {
        if(a[i] > a[j]) {
            temp = a[i];
            a[i] = a[j];
```

```
        a[j] = temp;
    }
}

printf("Sorted array:\n");
for(i = 0; i < n; i++)
    printf("%d ", a[i]);
break;

default:
    printf("Invalid Choice!");
}

return 0;
}
```

OUTPUT:

Enter number of elements: 5

Enter array elements:

5 3 1 4 2

1. Insertion

2. Deletion

3. Searching

4. Sorting

Enter your choice: 4

Sorted array:

1 2 3 4 5

Practical-8

AIM : Write a program to perform matrix operations such as addition, subtraction and multiplication using two-dimensional arrays.

CODE:

```
#include <stdio.h>

int main() {
    int a[10][10], b[10][10], c[10][10];
    int i, j, k, r1, c1, r2, c2, choice;

    printf("Enter rows and columns of Matrix A: ");
    scanf("%d%d", &r1, &c1);

    printf("Enter rows and columns of Matrix B: ");
    scanf("%d%d", &r2, &c2);

    printf("Enter elements of Matrix A:\n");
    for(i = 0; i < r1; i++)
        for(j = 0; j < c1; j++)
            scanf("%d", &a[i][j]);

    printf("Enter elements of Matrix B:\n");
    for(i = 0; i < r2; i++)
        for(j = 0; j < c2; j++)
            scanf("%d", &b[i][j]);

    printf("\n1. Addition\n2. Subtraction\n3. Multiplication\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice) {

        case 1:
```

```
if(r1 == r2 && c1 == c2) {
    printf("Resultant Matrix:\n");
    for(i = 0; i < r1; i++) {
        for(j = 0; j < c1; j++) {
            c[i][j] = a[i][j] + b[i][j];
            printf("%d ", c[i][j]);
        }
        printf("\n");
    }
} else {
    printf("Addition not possible.\n");
}
break;
```

case 2:

```
if(r1 == r2 && c1 == c2) {
    printf("Resultant Matrix:\n");
    for(i = 0; i < r1; i++) {
        for(j = 0; j < c1; j++) {
            c[i][j] = a[i][j] - b[i][j];
            printf("%d ", c[i][j]);
        }
        printf("\n");
    }
} else {
    printf("Subtraction not possible.\n");
}
break;
```

case 3:

```
if(c1 == r2) {
    printf("Resultant Matrix:\n");
    for(i = 0; i < r1; i++) {
        for(j = 0; j < c2; j++) {
            c[i][j] = 0;
            for(k = 0; k < c1; k++) {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}
```

```
        printf("%d ", c[i][j]);
    }
    printf("\n");
}
} else {
    printf("Multiplication not possible.\n");
}
break;

default:
    printf("Invalid Choice!");
}

return 0;
}
```

OUTPUT:

Enter rows and columns of Matrix A: 2 2

Enter rows and columns of Matrix B: 2 2

Enter elements of Matrix A:

1 2

3 4

Enter elements of Matrix B:

5 6

7 8

1. Addition

2. Subtraction

3. Multiplication

Enter your choice: 1

Resultant Matrix:

6 8

10 12

Practical-9

AIM : Write a program to perform string operations such as length, copy, concatenation and comparison using built-in string functions. _

CODE:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[100], str2[100];

    printf("Enter first string: ");
    scanf("%s", str1);

    printf("Enter second string: ");
    scanf("%s", str2);

    // Length
    printf("\nLength of first string = %lu\n", strlen(str1));
    printf("Length of second string = %lu\n", strlen(str2));

    // Copy
    char copy[100];
    strcpy(copy, str1);
    printf("Copied string = %s\n", copy);

    // Concatenation
    char concat[200];
    strcpy(concat, str1);
    strcat(concat, str2);
    printf("Concatenated string = %s\n", concat);

    // Comparison
    if (strcmp(str1, str2) == 0)
        printf("Both strings are Equal.\n");
    else
        printf("Both strings are Not Equal.\n");

    return 0;
}
```

OUTPUT:

```
Enter first string: Hello
Enter second string: World
Length of first string = 5
Length of second string = 5
Copied string = Hello
Concatenated string = HelloWorld
Both strings are Not Equal.
```

Practical-10

AIM: Write a program to perform string manipulation without using built-in string functions.

CODE:

```
#include <stdio.h>

int main() {
    char str1[100], str2[100], copy[100], concat[200];
    int i, j, len = 0, equal = 1;

    printf("Enter first string: ");
    scanf("%s", str1);

    printf("Enter second string: ");
    scanf("%s", str2);

    // Find length of first string
    while (str1[len] != '\0')
        len++;

    printf("\nLength of first string = %d\n", len);

    // Copy string
    i = 0;
    while (str1[i] != '\0') {
        copy[i] = str1[i];
        i++;
    }
    copy[i] = '\0';
    printf("Copied string = %s\n", copy);

    // Concatenate strings
    i = 0;
    while (str1[i] != '\0') {
        concat[i] = str1[i];
        i++;
    }

    j = 0;
```

```
while (str2[j] != '\0') {
    concat[i] = str2[j];
    i++;
    j++;
}
concat[i] = '\0';

printf("Concatenated string = %s\n", concat);

// Compare strings
i = 0;
while (str1[i] != '\0' || str2[i] != '\0') {
    if (str1[i] != str2[i]) {
        equal = 0;
        break;
    }
    i++;
}

if (equal)
    printf("Both strings are Equal.\n");
else
    printf("Both strings are Not Equal.\n");

return 0;
}
```

OUTPUT:

```
Enter first string: Hello
Enter second string: World
Length of first string = 5
Copied string = Hello
Concatenated string = HelloWorld
Both strings are Not Equal.
```

Practical-11

AIM : Develop a program that calculates factorial and Fibonacci series using user-defined functions and recursive functions.

CODE:

```
#include <stdio.h>

// Function to calculate factorial using recursion
int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

// Function to calculate Fibonacci using recursion
int fibonacci(int n) {
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n, i;

    printf("Enter a number: ");
    scanf("%d", &n);

    // Factorial
    printf("Factorial of %d = %d\n", n, factorial(n));

    // Fibonacci Series
    printf("Fibonacci Series up to %d terms:\n", n);
    for (i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }

    return 0;
}
```

Output:

Enter a number: 5

Factorial of 5 = 120

Fibonacci Series up to 5 terms:

0 1 1 2 3

Practical-12

AIM: Design a student record system using structures to store and display information such as roll number, name, marks and grade.

CODE:

```
#include <stdio.h>

struct Student {
    int rollNo;
    char name[50];
    float marks;
    char grade;
};

int main() {
    struct Student s;

    // Input
    printf("Enter Roll Number: ");
    scanf("%d", &s.rollNo);

    printf("Enter Name: ");
    scanf("%s", s.name);

    printf("Enter Marks: ");
    scanf("%f", &s.marks);

    printf("Enter Grade: ");
    scanf(" %c", &s.grade);

    // Output
    printf("\n--- Student Record ---\n");
    printf("Roll Number : %d\n", s.rollNo);
    printf("Name      : %s\n", s.name);
    printf("Marks      : %.2f\n", s.marks);
    printf("Grade      : %c\n", s.grade);

    return 0;
}
```

OUTPUT:

Enter Roll Number: 101

Enter Name: Rahul

Enter Marks: 89.5

Enter Grade: A

--- Student Record ---

Roll Number : 101

Name : Rahul

Marks : 89.50

Grade : A

Practical-13

AIM: Write a program to demonstrate structures, nested structures, arrays within structures and unions.

CODE:

```
#include <stdio.h>
// Nested Structure
struct Address {
    char city[30];
    char state[30];
};
// Structure with Array and Nested Structure
struct Student {
    int rollNo;
    char name[30];
    int marks[3];    // Array within structure
    struct Address addr; // Nested structure
};

// Union
union Data {
    int i;
    float f;
    char str[20];
};

int main() {
    struct Student s;
    union Data d;

    // Input Student Details
    printf("Enter Roll Number: ");
    scanf("%d", &s.rollNo);

    printf("Enter Name: ");
    scanf("%s", s.name);

    printf("Enter Marks in 3 Subjects: ");
    scanf("%d%d%d", &s.marks[0], &s.marks[1], &s.marks[2]);

    printf("Enter City: ");
    scanf("%s", s.addr.city);
```

```
printf("Enter State: ");
scanf("%s", s.addr.state);

// Display Student Details
printf("\n--- Student Details ---\n");
printf("Roll Number : %d\n", s.rollNo);
printf("Name      : %s\n", s.name);
printf("Marks      : %d %d %d\n",
       s.marks[0], s.marks[1], s.marks[2]);
printf("City       : %s\n", s.addr.city);
printf("State      : %s\n", s.addr.state);

// Union Demonstration
d.i = 100;
printf("\nUnion Integer : %d\n", d.i);

d.f = 25.75;
printf("Union Float  : %.2f\n", d.f);

return 0;
}
```

OUTPUT:

Enter Roll Number: 101

Enter Name: Rahul

Enter Marks in 3 Subjects: 85 90 88

Enter City: Delhi

Enter State: Delhi

--- Student Details ---

Roll Number : 101

Name : Rahul

Marks : 85 90 88

City : Delhi

State : Delhi

Union Integer : 100

Union Float : 25.75

Practical-14

AIM: Write a program that uses pointers with arrays and functions and demonstrates dynamic memory allocation using malloc(), calloc(), realloc() and free().

CODE:

```
#include <stdio.h>
#include <stdlib.h>

// Function to display array elements using pointers
void display(int *arr, int n) {
    int i;
    printf("Array Elements: ");
    for(i = 0; i < n; i++) {
        printf("%d ", *(arr + i));
    }
    printf("\n");
}

int main() {
    int *arr, n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    // malloc()
    arr = (int *)malloc(n * sizeof(int));
    if(arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
        scanf("%d", (arr + i));

    printf("\nUsing malloc():\n");
    display(arr, n);

    // calloc()
    int *arr2 = (int *)calloc(n, sizeof(int));
    if(arr2 == NULL) {
        printf("Memory allocation failed.\n");
    }
```

```
    free(arr);
    return 1;
}

printf("\nUsing calloc() (initialized to 0):\n");
display(arr2, n);

// realloc()
printf("\nEnter new size: ");
scanf("%d", &n);

arr = (int *)realloc(arr, n * sizeof(int));
if(arr == NULL) {
    printf("Memory reallocation failed.\n");
    free(arr2);
    return 1;
}

printf("Enter %d elements:\n", n);
for(i = 0; i < n; i++)
    scanf("%d", (arr + i));

printf("\nAfter realloc():\n");
display(arr, n);

// free()
free(arr);
free(arr2);

printf("\nMemory deallocated successfully.\n");

return 0;
}
```

OUTPUT:

Enter number of elements: 3

Enter 3 elements:

10 20 30

Enter new size: 5

Enter 5 elements:

10 20 30 40 50

Using malloc():

Array Elements: 10 20 30

Using calloc() (initialized to 0):

Array Elements: 0 0 0

After realloc():

Array Elements: 10 20 30 40 50

Memory deallocated successfully.

Practical-15

AIM: Write a program to read from and write to files using file handling functions such as `fopen()`, `fclose()`, `fprintf()`, `fscanf()`, `fread()` and `fwrite()`.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    FILE *fp;
    struct Student s = {101, "Rahul", 89.5}, s1;

    // Writing text using fprintf()
    fp = fopen("student.txt", "w");
    fprintf(fp, "%d %s %.2f", s.roll, s.name, s.marks);
    fclose(fp);

    // Reading text using fscanf()
    fp = fopen("student.txt", "r");
    fscanf(fp, "%d %s %f", &s1.roll, s1.name, &s1.marks);
    fclose(fp);

    printf("Data read using fscanf():\n");
    printf("Roll: %d\nName: %s\nMarks: %.2f\n",
        s1.roll, s1.name, s1.marks);

    // Writing binary data using fwrite()
    fp = fopen("student.dat", "wb");
    fwrite(&s, sizeof(struct Student), 1, fp);
    fclose(fp);

    // Reading binary data using fread()
    fp = fopen("student.dat", "rb");
    fread(&s1, sizeof(struct Student), 1, fp);
    fclose(fp);

    printf("\nData read using fread():\n");
    printf("Roll: %d\nName: %s\nMarks: %.2f\n",
        s1.roll, s1.name, s1.marks);

    return 0;
}
```

OUTPUT:

Data read using fscanf():

Roll: 101

Name: Rahul

Marks: 89.50

Data read using fread():

Roll: 101

Name: Rahul

Marks: 89.50